7. Computer Architecture

Stakeholder: **Jaidev Khalane**, **Arjun Anand Mallya 23110039**

There are two main parts in the Computer Architecture Problem Statements, the first one is Satellite Based Low-Power Onboard Computer & OS for Space Debris Avoidance for MIPS32 based Assembly and the second one is Hardware Acceleration of ML Activations on an Integer RISC-V Core based on Verilog.

Total Points: 200
Team Size: 1-2

Part	Language	Marks	Contact
Part 1. Satellite Based Low-Power Onboard Computer & OS for Space Debris Avoidance	Assembly	100 (60% weightage)	**Jaidev Khalane**
Part 2. Hardware Acceleration of ML Activations on an Integer RISC-V Core	Verilog	100 (40% weightage)	**Arjun Anand Mally...**

Minimum Threshold Score: NONE
Top 2 Teams will be rewarded.

Final Submission for Evaluation:

- Source Code
- Documentation
- 10-15 minute Presentation (In-Person)

In Person Presentation is mandatory for being eligible for the Prizes.

Important Notes:
Since there are not many strict constraints on the expected outcome, the participants have the flexibility to implement the requirements in any form they want. Therefore, it would be the responsibility of the participants to create the test cases based on the guidelines (conditions to be modelled would be provided by us) that would be provided during the Hackathon.

For Example:

Part 1
Initial Position of Satellite: _
Angular Velocity of Satellite : _
Radius of Orbit of Satellite: _
Number of Debris Objects: _
For Every Debris Object:
    Positions of the Debris Objects: _
    Velocity of Debris Objects: _
In Polar Coordinate format with an assumption of a 2D setup with Earth at the origin.

OR

Part 2
A .txt file containing the input numbers (Integers - 32 bits).

The test cases would be provided and the participants should modify them based on their specific models.

There would be two rounds of test cases released. The first round would be released after 12 hours of the Hackathon and the second round would be given after 35 hours.

PART 1. Satellite Based Low-Power Onboard Computer & OS for Space Debris Avoidance

Contact: Jaidev Khalane

Image Credits: NASA - Hubble Space Telescope

Refer to This Link: HackRush 2026: Computer Architecture Problem Statement

1. High Level Problem Statement

Design and implement, in C and (preferably) assembly, a minimal onboard computer system for a cubesat-class satellite equipped with a simple 32-bit RISC CPU (MIPS-like), small memory, no FPU, and strict power constraints.

The onboard system must:

- Maintain a model of the satellite's orbit and a catalog of nearby space debris.
- Periodically predict potential close approaches (conjunctions) with debris.
- Decide simple avoidance maneuvers when necessary.
- Run on top of a tiny OS-like runtime that:
    - Schedules multiple periodic and aperiodic tasks.

    - Manages memory explicitly without an MMU.
    - Manages power states (active vs sleep) to stay within an energy budget.

The core goal is to embed computer architecture and organisation principles (ISA, pipeline timing, memory hierarchy, energy model) and operating systems concepts (scheduling, resource management, power-aware policies) into an end-to-end, low-compute software design.

Target Duration: 36-hours

Team Size: 2

2. Motivation

2.1. Real-World Background

- Modern space missions, including cubesats and small satellites, are constrained by:
    - Limited onboard compute: microcontrollers with low frequency, few registers, limited RAM, and no floating-point hardware.
    - Strict energy budgets: solar panels and small batteries require aggressive duty-cycling of CPU and radios.
    - Increasing space debris density in Low Earth Orbit (LEO), increasing collision risks for small satellites.
- High-precision orbit propagation and collision avoidance are often done on the ground, but:
    - Communication windows are intermittent.
    - Latency to ground can be too large for last-minute avoidance.
    - A minimal onboard autonomy layer is desirable to perform local risk assessment and basic maneuvers.
- This problem ties directly to emerging research on space debris tracking and collision avoidance.

2.2. Learning Objectives

- Make participants reason concretely about a CPU model:
    - How many cycles do their algorithms need?
    - How do pipeline hazards, memory access patterns, and lack of an FPU impact design?
- Force OS-level thinking:
    - Task scheduling and priorities.
    - Memory layout without an MMU.
    - Power-state transitions and energy accounting.
- Move beyond pure algorithm design into whole-system engineering under constraints.

Participants must integrate:

- Computer Architecture & Organisation: Load/store ISA, pipeline timing, cache, fixed-point arithmetic.
- Operating Systems: Scheduling, timing, resource management, power-aware policies.
- Application Domain: Simplified orbital dynamics and conjunction analysis.

3. Platform and Abstract Hardware Model

Participants will not program real hardware; instead, they should build an emulator targeting a specified abstract CPU + memory + power model. This model is used for evaluation and must be respected in all design choices.

3.1 CPU Model

- Architecture: 32-bit MIPS-like RISC
    - 32 general-purpose registers (R0 hard-wired to 0).
    - No hardware floating-point unit (no FPU).
    - Register-based calling convention (similar to standard MIPS).

- Pipeline:
    - Classic 5-stage pipeline: IF, ID, EX, MEM, WB.
    - Hazards:
        - Load-use hazard incurs 1-cycle stall if not avoided.
        - Taken branches incur 1-cycle penalty (flush).
        - Multiplies/divides are multi-cycle (configurable cost).
- Instruction timing (example constants):
    - ALU ops (ADD/SUB/AND/OR/XOR, shifts): 1 cycle.
    - Branch (BEQ/BNE) + pipeline penalty if taken: 1 + 1 cycles.
    - Load: 1 cycle + potential memory penalty.
    - Store: 1 cycle + potential memory penalty.
    - Multiply: 3 cycles.

3.2 Memory Model

- Main memory:
  - Total RAM: 128 KB (Based on requests, increasing this may be considered).
  - Code + data must fit within this limit.
- Memory regions (no MMU):
  - .text: program code.
  - .data/.bss: global data, static buffers.
  - Stack: downward-growing region per task.
- Optional cache (simplified):
  - 4 KB "fast memory" region with no extra access penalty.
  - All other accesses cost an extra +3 cycles (simulating off-chip RAM).
  - Teams can choose to place critical data structures.

3.3 Power Model

- Power states:
  - Active: CPU is running instructions.
  - Sleep: CPU clock gated, only timer/interrupt logic active.
- Costs:
  - Active: 20 mW (per active cycle).
  - Sleep: 0.5 mW (per unit time).
  - Radio active (TX/RX): +50 mW on top of current state.
- Energy accounting:
  - Total energy $E=\sum P_{state} \times \Delta t$.
  - The teams with the lowest energy will be prioritized.

Teams do not implement hardware; they must conform to this model to develop the emulator upon which the next phase (software stack) is based. Since the emulator instructions will be similar to MIPS32, the participants can compile the software part to MIPS and then use the direct mapping between the MIPS instructions and their custom instructions to convert the MIPS compiled C code to their own custom emulator's assembly. If any team wants to skip the development of the emulator completely, they are free to do so given that they compile to the MIPS32 Architecture based processor. However, the above constraints must still be satisfied.

4. Software Requirements

The software part should consist of:

- Core logic in C, with optional hand-tuned assembly for critical kernels.

- A minimal OS-like runtime:
    - Task scheduler.
    - Basic memory management scheme.
    - Power management and state tracking.
- An application layer implementing:
    - Orbit and debris state update.
    - Collision risk prediction.
    - Maneuver planning.
    - Telemetry formatting.

4.1 Task Model

Define the following tasks (at minimum):

1. T1: Orbit Propagation Task
    - Periodic (every 5 s of simulated time).
    - Updates satellite's orbital state using a simplified model (two-body approximation or linearized update).
    - Updates debris catalog entries when new uplink messages are available.
2. T2: Collision Risk Evaluation Task
    - Periodically (every 10 s), data will be sent from Earth.
    - For each debris object:
        - Predict relative position over a short window.
        - Compute a simple proximity metric (e.g., minimum distance over N time steps).
    - Classify risk: SAFE / WATCH / HIGH-RISK.
3. T3: Maneuver Planning Task
    - Aperiodic, triggered when T2 finds HIGH-RISK objects.
    - Evaluates a small discrete set of candidate maneuvers:
        - No maneuver.
        - Along-track $\pm\Delta v$ (e.g., +3 cm/s, -3 cm/s).
        - Radius $\pm\Delta r$ (eg., +1m, -1m) in orbital radius.
    - Uses quick propagation to estimate which candidate increases separation the most.
    - Outputs a candidate maneuver command and updates its own orbit.
4. T4: Telemetry & Logging Task
    - Periodic (every 20 s).
    - Packs:
        - Latest orbit state.
        - List of HIGH-RISK objects.
        - Planned maneuvers.

        - Resource usage (CPU, energy estimates).
    - Marks radio as active during transmission (energy penalty).
5. T5: Idle / Power Management Task
    - Runs when no other tasks are ready.
    - Puts CPU into sleep state (simulated).
    - Wakes up on timer events or simulated interrupts.
6. T6: Data Transmission Stage
    - Transmits some weather data back to the Earth. Simulated by a custom step.

Teams must implement a scheduler that respects these periodicities and task priorities.

4.2 Scheduler and OS Runtime

- Scheduler type:
    - At least priority-based preemptive (via simulated timer interrupt), or
    - Priority-based cooperative with explicit yield points and strict reasoning about deadlines.
- Task representation:
    - Task Control Blocks (TCBs) with:
        - Task state (READY/RUNNING/BLOCKED).
        - Priority.
        - Next release time (for periodic tasks).
        - Stack pointer and register context (if implementing preemption).
- Timing:
    - A global simulated clock.
    - Timer tick (e.g., 10 ms tick) that drives periodic tasks and wakeups.

4.3 Memory Management

- No full dynamic memory requirement, but teams must:
    - Partition memory into:
        - Global data region.
        - Per-task stacks with bounded size.
        - Static buffers for debris catalog and logs.
    - Avoid unbounded recursion and dynamically sized allocations in critical paths.
- Optional:
    - A simple fixed-size allocator for non-critical tasks (e.g., logging).
- Emphasis:

- Correct, deterministic use of memory (no stack overflows, no buffer overruns under normal loads)

5. Dataset / Scenario Specification

There is no typical tabular dataset; instead, evaluation is driven by simulated circular / elliptical / hyperbolic orbital scenarios consistent with realistic space debris behavior. The teams have the flexibility to implement the overall system and generate the dataset satisfying the constraints posed in this document.

5.1 Scenario Files

Each scenario is a text or binary file containing:

1. Simulation parameters
    - Simulation horizon (e.g., 1 hour - for evaluation, we will scale it to a few minutes).
    - Decision time step for collision evaluation (e.g., 10 s).
    - Energy budget for that horizon.
2. Satellite initial state
    - Assume the Satellite orbit to be circular.
    - The initial orbit radius and angular velocity will be provided
3. Debris catalog
    - N objects (e.g., N = 50-200).
    - For each object, the following information will be sent in every 10 seconds:
        - Current position (polar) and velocity (radial and tangential).
        - Size.
        - A unique ID.
    - The implementation can perform a linear interpolation within a safe margin of the size of the debris until the updated data is obtained.
    - Expect the dataset to be in json format, consisting of the timestamp as the key and the later parameters as a nested object. Teams will be provided with the json file content whose keys would be as given above.
4. Uplink update schedule
    - Times when new or updated catalog entries arrive.
    - Some updates may contain:
        - New objects.
        - Revised states of existing objects.

Initial Test Cases will be provided after 12 hours of the Hackathon's Initiation. They may require some modifications based on the specifications of the implementation.

6. Expected Outcomes from Teams

Each team must deliver:

6.1 Executable and Build Scripts

- build.sh:
  - Compiles the C/assembly code for the reference environment.
- run.sh:
  - Interface: ./run.sh scenario_X.json > output_X.log.
  - Reads scenario, runs simulation, and prints outputs in specified format.

6.2 Behavior and Outputs at Runtime

For each scenario, the program must:

1. Respect the CPU and power model:
   - Track cycles used in each task and decision window.
   - Track energy usage based on active/sleep/radio periods.
2. Produce:
   - Collision-risk reports:
     - Time-stamped list of debris objects classified as SAFE/WATCH/HIGH-RISK.
   - Maneuver recommendations:
     - For each HIGH-RISK event, a maneuver (or explicit "no maneuver") with:
       - Time to execute.
       - Δv vector (in some abstracted form).
   - Resource reports:
     - CPU utilization (approximate).
     - Total cycles, per-task cycles.
     - Estimated energy usage and current battery level.

6.3 Documentation

A short design report (2-4 pages) must include:

- Emulator design with hardware model assumptions (you can restate the provided model and any additional assumptions).
- OS/runtime design:
  - Scheduler type, task priorities, timing and deadline reasoning.

    - Power-management policies.
- Architecture-aware optimizations:
    - Where fixed-point arithmetic is used and why.
    - Where assembly is used and why.
    - Data layout decisions (memory locality, scratchpad usage).
- Known limitations and future improvement

7. Evaluation Method

Evaluation is a two-stage process:

1. Execution based scoring on the developed scenarios that the teams would implement based on the test case guidelines to be provided.
2. Qualitative review with 10-15 minute Presentation for design quality and code clarity.

7.1 Evaluation Pipeline

For each team, we will run a few test case scenarios and analyse the performance of the developed project based on the above metrics and the extent to which they satisfy the requirements. Metrics are aggregated across all scenarios to generate a normalised numeric score based on the test cases made.

7.2 Collision-Risk Evaluation

For each scenario:

- For each debris object and time window:
    - Compare the team's classification:
        - SAFE / WATCH / HIGH-RISK
    - Against ground-truth risk based on reference simulation.
- A "dangerous" classified event: minimum distance below a danger threshold.
- "High-Risk" ground truth label: event where actual minimum distance is below a danger threshold.

Compute:

- True Positive (TP): Team labels HIGH-RISK when ground truth is dangerous.
- False Negative (FN): Team does not label HIGH-RISK when ground truth is dangerous.

- False Positive (FP): Team labels HIGH-RISK when ground truth is safe.

Metrics:

- Recall =TP / (TP+FN).
- Precision =TP / (TP+FP).

Scoring:

- High recall is critical:
  - Missing a real collision (FN) is heavily penalized.
- Precision matters:
  - Too many false alarms (FP) penalized but less severely.

The collision-risk component score is derived from these metrics.

7.3 Maneuver Effectiveness Evaluation

For each recommended maneuver:

1. Simulate two trajectories using the reference model:
   - Baseline: no maneuver.
   - With recommended maneuver at stated time.
2. Compute:
   - Baseline minimum distance d^{base}_{min}.
   - Maneuver minimum distance $d^{maneuver}_{min}$.

Score per maneuver:

- If $d^{maneuver}_{min} > d^{base}_{min}$: positive score, proportionate to improvement.
- If $d^{maneuver}_{min} \leq d^{base}_{min}$: small or negative score (wasted or harmful maneuver).

7.4 Compute and Timing Efficiency

During execution, teams must:

- Maintain a per-task cycle counter:
  - Each function increments counters using known cost per instruction or wrapper macros.
- Or, run under a provided cycle-estimating wrapper that counts instructions.

Evaluation:

- For each decision window (e.g., one full T2 + T3 cycle):
    - Check if total cycles $C_{window} \leq C_{max}$.
    - Overrun events are counted.
- Aggregate:
    - Average and worst-case cycles per window.
- Score:
    - Perfect adherence (no overruns): full points.
    - Occasional overruns: proportional penalty.
    - Frequent overruns: larger penalty.

7.5 Energy and Duty-Cycling Behavior

Using the power model:

- For each simulated time step, track state:
    - Active vs sleep.
    - Radio usage.
- Integrate energy:
    - Compute total energy E_{total}.
- Compare to scenario's energy budget E_{budget}.

Scoring:

- Among teams that meet budgets:
    - Rank by energy efficiency (less energy → more bonus points).
- Additionally:
    - Track fraction of time in sleep vs active:
        - A higher proportion of sleep, subject to meeting deadlines, is rewarded.

8. Qualitative Evaluation

After computing quantitative scores, we will review:

8.1 Systems Design Quality

- Proper modularization:
  - Clear separation between scheduler, tasks, hardware abstraction, and application logic.
- Clear interface definitions and minimal coupling.
- Thoughtful design of priorities and timing:
  - Justified in the design document.

8.2 Architecture-Aware Implementation

- Use of fixed-point arithmetic instead of floating point.
- Use of scratchpad/fast memory for hotspot data.
- Effective use of assembly for:
  - Inner loops of propagation or collision checking.
  - Operations that benefit from manual instruction scheduling or loop unrolling.
- Comments explaining ISA-level decisions (e.g., branch patterns, pipeline concerns).

8.3 Documentation & Clarity

- Design report clearly:
  - Explains assumptions.
  - Justifies choices.
  - States known limitations.

9. Grading Scheme (Out of 100)

9.1 Quantitative - 50 marks

1. Collision-Risk Detection Quality - 35 marks
    - Recall and Precision
2. Maneuver Effectiveness - 15 marks
    - Average improvement in minimum distance across relevant encounters.
3. Compute & Timing Efficiency - 15 marks
    - Number of cycles limit per decision window (lesser would be rewarded more).
4. Energy & Duty-Cycling - 10 marks
    - Energy budget respected (lower budget will be rewarded).
    - Percentage time in sleep state (while meeting correctness/timing).
5. Robustness - 5 marks
    - No crashes/hangs
    - Sensible behavior on corrupted input, no wild outputs.
    - Failures reduce marks accordingly.

9.2 Qualitative - 50 marks

6. Systems & OS Design Quality - 30 marks
    - Clear scheduler design (priorities, periods, deadlines explained).
    - Strong memory layout & task structuring.
    - Clean separation of concerns, minimal global state, understandable flow.
7. Architecture-Aware Implementation & Optimization - 15 marks
    - Meaningful use of fixed-point math.
    - Appropriate assembly optimizations in hotspots.
    - Evidence of reasoning about pipeline/memory behavior.
8. Documentation & Clarity - 5 marks
    - Complete, coherent design report.
    - Clear description of limitations & future work.

The Grading will be done RELATIVELY

10. Awards

In case of a tie, priority will be given to the teams that:

1. Build and run successfully on all basic scenarios.
2. Not exceed catastrophic limits:
   - 2.1. Extreme energy budget violation.
   - 2.2. Repeated hard timing failures.
3. Achieve minimum collision-risk recall (e.g., at least 60-70%).

11. Resources for Study

- **Computer Architecture:**
  - https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/pages/lecture-notes/
  - https://www.cse.iitd.ac.in/~srsarangi/archbooksoft.html
- **Operating Systems:**
  - https://ocw.mit.edu/courses/6-828-operating-system-engineering-fall-2012/pages/lecture-notes-and-readings/
  - https://pages.cs.wisc.edu/~remzi/OSTEP/
- **For Orbit Modelling:**
  - https://en.wikipedia.org/wiki/Simplified_perturbations_models

12. References

1. https://www.krasamo.com/low-power-embedded-systems/
2. https://www.stldigital.tech/blog/how-to-optimize-embedded-system-for-low-power-consumption/
3. https://runtimerec.com/low-power-design/
4. https://pmc.ncbi.nlm.nih.gov/articles/PMC10858253/
5. https://www.sciencedirect.com/science/article/abs/pii/S009457651730930X
6. https://arxiv.org/html/2506.0261